PRACTICAL NO.06

SUBMITTED BY:-
RAGAV SHARMA
PARTH KITCHLOO

▯ POLYMORPHISM IN JAVA

▯ Introduction to Polymorphism

Polymorphism is one of the main concepts of **Object-Oriented Programming (OOP)** in Java.

The word *Polymorphism* is derived from:

- **Poly** → Many
- **Morph** → Forms

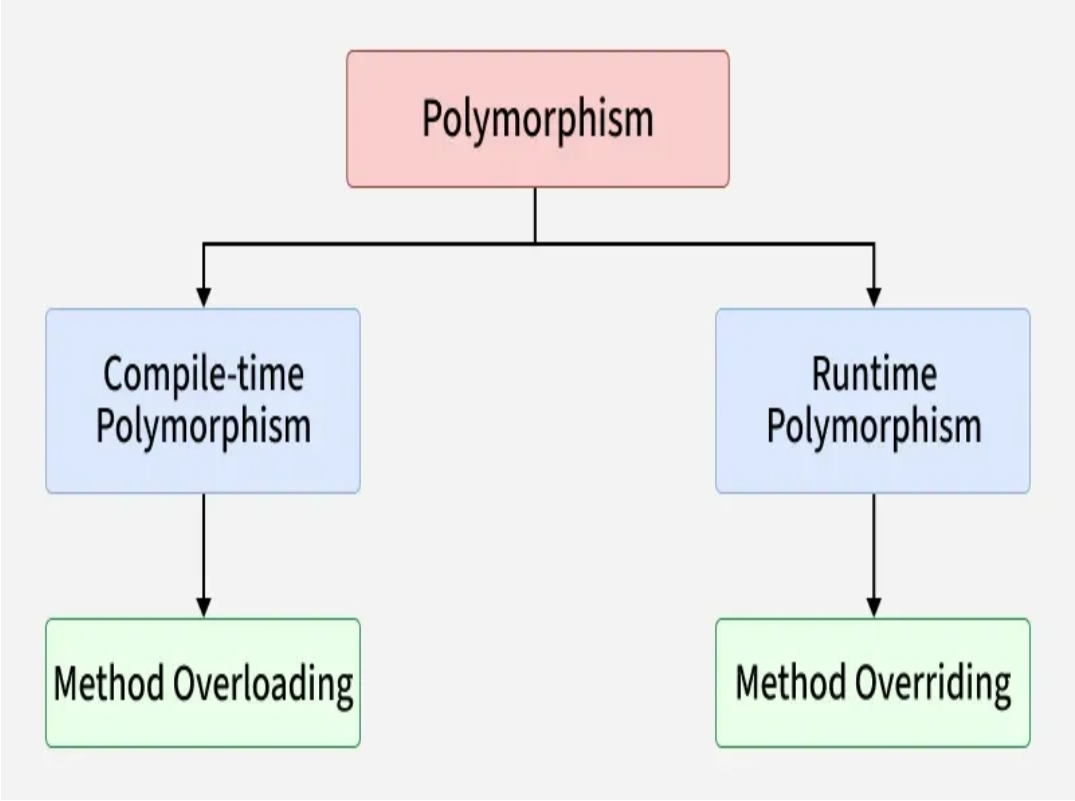
▯ So, Polymorphism means **"one name, many forms"**.

In Java, polymorphism allows a method, object, or operator to perform different actions depending on the situation.

▯ Types of Polymorphism in Java

Java supports two types of polymorphism:

1 **Compile-Time Polymorphism (Static Polymorphism)**
2 **Run-Time Polymorphism (Dynamic Polymorphism)**



1. Compile-Time Polymorphism (Method Overloading)

2. Runtime Polymorphism (Method Overriding)

Compile-Time Polymorphism

Also called **Method Overloading**.

✔ Achieved when multiple methods have the same name but different parameters.
✔ Decision is made at **compile time**.

▯ Method Overloading Rules

- Method name must be same.
- Parameters must be different (number or type).
- Return type alone cannot differentiate methods.

✅ General Syntax

```java
class ClassName {

    returnType methodName(parameter1) {
        // code
    }

    returnType methodName(parameter1, parameter2) {
        // code
    }

    returnType methodName(differentDataType parameter) {
        // code
    }
}
```

Run-Time Polymorphism

```java
class ParentClass {

    void methodName() {
        // parent code
    }
}

class ChildClass extends ParentClass {

    @Override
    void methodName() {
        // child code
    }
}
```

Also called **Method Overriding**.

✔ Achieved through **Inheritance**.
✔ Method call is decided at **runtime**.
✔ Based on object type, not reference type.

⍰ Rules for Method Overriding

- Same method name.
- Same parameters.

- Must be in parent and child class.
- Access modifier cannot be more restrictive.
- Cannot override static or final methods.

Dynamic Method Dispatch

Dynamic method dispatch is the mechanism by which:

✔ A call to an overridden method
✔ Is resolved at runtime
✔ Based on object type

Example:

Java

```java
Animal obj = new Dog();
obj.sound();
```

✅ Syntax

Java

```java
ParentClass reference;
reference = new ChildClass();
reference.methodName();
```

Even though reference is `Animal`, the object is `Dog`, so `Dog` method is executed.

Overloading vs Overriding

Feature	Overloading	Overriding
Class	Same class	Parent & Child
Parameters	Must differ	Must be same
Binding	Compile time	Runtime
Inheritance	Not required	Required
Keyword	No special keyword	@Override recommended

▯ Advantages of Polymorphism

✔ Code reusability
✔ Flexibility
✔ Extensibility
✔ Improves readability
✔ Supports dynamic binding

▯ Real Life Example

Think of a **Remote Control**:

- One button → Different functions (TV, AC, Fan)
- Same interface → Different behavior

That is polymorphism.

Disadvantages of Polymorphism

1. Slight performance overhead
2. Harder debugging
3. Can increase complexity if misused

Upcasting and Downcasting

✅ Upcasting

When a child class object is referred by parent class reference.

It is:

- Automatic
- Safe
- Used in runtime polymorphism

Example:

```java
Animal a = new Dog();
```

✅ Downcasting

When parent reference is converted into child reference.

It is:

- Explicit
- Risky if wrong type
- Used to access child-specific methods

Example:

```java
Dog d = (Dog) a;
```

instance of Operator

Used to check object type before downcasting.

It prevents:
✔ ClassCastException

Example:

```java
if(a instanceof Dog)
```

⍰ Polymorphism with Interfaces

Java also supports runtime polymorphism using interfaces.

✅ Interface Syntax

```java
interface InterfaceName {
    void methodName();
}

class ClassName implements InterfaceName {
    public void methodName() {
        // implementation
    }
}
```

⍰ Polymorphism with Abstract Class

✓ Abstract Class Syntax

```java
abstract class ClassName {

    abstract void methodName();   // abstract method

    void normalMethod() {
        // normal method
    }
}

class ChildClass extends ClassName {

    void methodName() {
        // implementation
    }
}
```

Static Binding vs Dynamic Binding

Feature	Static Binding	Dynamic Binding
Time	Compile Time	Runtime
Type	Overloading	Overriding
Speed	Faster	Slightly slower
Inheritance	Not required	Required

🔷 Compile-Time vs Run-Time Polymorphism

Feature	Compile-Time	Run-Time
Method	Overloading	Overriding
Binding	Early Binding	Late Binding
Inheritance Required	No	Yes
Decision Time	Compile Time	Runtime

🔑 Key Points

- Polymorphism improves program flexibility.
- Overloading → Same class.
- Overriding → Parent & Child class.
- Achieved using inheritance and interfaces.

🔑 Conclusion

Polymorphism is a powerful OOP concept that allows:

✔ Same method name
✔ Different implementations
✔ Flexible code design
✔ Runtime decision making

It is widely used in:

- Enterprise applications
- Frameworks (Spring, Hibernate)
- GUI applications
- Banking systems
- Game development